TEXAS A&M UNIVERSITY

Logic

CSCE 222: Discrete Structures for Computing

Outline

- Propositional logic
- Logical operators
- Propositional equivalences

What is logic?

Logic is the basis of all mathematical reasoning

Logic symbols

Symbol	Symbol Name	Meaning	Example
$\cdot$	and	and	$x \cdot y$
$\wedge$	caret / circumflex	and	$x \wedge y$
&	ampersand	and	$x \& y$
+	plus	or	$x + y$
$\vee$	reversed caret	or	$x \vee y$
|	vertical line	or	$x \mid y$
x'	single quote	not - negation	x'
	bar	not - negation	
$\neg$	not	not - negation	$\neg x$
!	exclamation mark	not - negation	$!x$
$\oplus$	circled plus / oplus	exclusive or - xor	$x \oplus y$
~	tilde	negation	$\sim x$
$\Rightarrow$	implies		
$\Leftrightarrow$	equivalent	if and only if (iff)	
$\forall$	for all		
$\exists$	there exists		
$\nexists$	there does not exists		
$\therefore$	therefore		
$\because$	because / since		

Propositional logic

A proposition is a declarative sentence that is either true or false.

Examples:

- Python is a compiled language
- A binary tree with three nodes has exactly two leaf nodes
- A stack follows the Last-In-First-Out (LIFO) principle
- Inserting an element at the beginning of an array is O(1) time complexity
- Linux was created by Linus Torvalds
- JavaScript was created before the Java programming language

Propositional logic

A proposition is a declarative sentence that is either true or false.

Examples:

- Python is a compiled language
- A binary tree with three nodes has exactly two leaf nodes
- A stack follows the Last-In-First-Out (LIFO) principle
- Inserting an element at the beginning of an array is O(1) time complexity
- Linux was created by Linus Torvalds
- JavaScript was created before the Java programming language

Propositional logic

We use letters to represent propositional variables (e.g. p, q, r, s)

Examples:

p: the student is a CSE major

q: the lecture starts at 8 am

Compound propositions

Compound propositions are formed **combining** existing propositions using logical **operators**.

Basic components of a logic statement are: and, or, not

and	$p \wedge q$	p and q	**conjunction** of p and q
or	$p \vee q$	p or q	**disjunction** of p and q
not	$\sim q$	not q	**negation** of q
xor	$p \oplus q$	p xor q	**exclusive or** of p and q
cond.	$p \rightarrow q$	p implies q	**conditional**
bicond.		p iff q	**biconditional**

Compound propositions

"The server is running and accepting connections"

Let p = "the server is running" and q = "the server is accepting connections"

We can translate this statement as: $p \wedge q$

"The code throws an exception or returns null"

Let r = "the code returns null". Then we have: $q \vee r$

(where q = "the code throws an exception")

"The code does not return null"

becomes $\sim r$

Truth value

A proposition is either **true** or **false**.

We denote the truth value for a proposition as follows:

T = True proposition

F = False proposition

Truth table

A **truth table** displays the **truth values** for <u>all</u> possible combinations of truth values for the proposition variables.

Truth tables are helpful for evaluating compound propositions!

Creating a truth table

Example: Create a truth table for: $p \wedge q$

For each variable, we have 2 possible truth values (T, F)

How many rows are there in the table?

(Combinations of truth values for p, q, r)

Rule: for **n** variables, there are $\mathbf{2^n}$ possible truth value combinations

Conjunction (AND)

If **both** p and q are true, then $p \wedge q$ is **true**; otherwise, $p \wedge q$ is false.

Truth table for conjunction:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

Disjunction (OR)

If **at least one** of p and q is **true**, then $p \vee q$ is true; otherwise, $p \vee q$ is false.

Truth table for disjunction:

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Negation (NOT)

If p is true, then ~p is false
If p is false, then ~p is true

Truth table for negation:

p	~p
T	F
F	T

Exclusive-OR (XOR)

Outputs true only when p and q differ

XOR: $(p \wedge \sim q) \vee (\sim p \wedge q)$

Truth table for XOR:

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

Conditional Statements

General form: **if** p, **then** q

Denoted as $p \longrightarrow q$ "p **implies** q"

p = **hypothesis**

q = **conclusion**

What are the truth values for p, q, $p \longrightarrow q$?

Conditional Statements

Example: "If the code doesn't compile, then the program won't execute"

hypothesis p: the code doesn't compile

conclusion q: the program won't execute

Conditional Statements

1. Assume both p and q are true.

"The code doesn't compile, and the program won't execute"

The conditional statement is true

2. Next, let p be true and q be false

"The code doesn't compile, and the program does execute"

The conditional statement is false

Conditional Statements

3. Next, let p be false and q be true

"The code does compile, and the program won't execute"

The conditional statement is true

4. Next, let p and q both be false

"The code does compile, and the program does execute"

The conditional statement is true

Conditional Statements

The truth table for $p \longrightarrow q$ is:

p	q	$p \longrightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Converse, Inverse, Contrapositive

$\neg p \rightarrow \neg q$ is the **inverse** of $p \rightarrow q$

$q \rightarrow p$ is the **converse** of $p \rightarrow q$

$\neg q \rightarrow \neg p$ is the **contrapositive** of $p \rightarrow q$

Converse, Inverse, Contrapositive

Example: “If the password is correct, then you can login”

Biconditional Statements

Notation: $p \leftrightarrow q$
Meaning: “p if and only if q”, “p iff q”

True if p and q have the same truth value (both true or both false), false otherwise.

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

Combining Logical Operators

We can combine propositions with logical operators to make complex logical expressions.

We can use parentheses to override the default precedence.

e.g. $\neg p \wedge q \rightarrow r$ **means** $((\neg p) \wedge q) \rightarrow r$

Order of precedence	Operator
1	$\neg$
2	$\wedge$
3	$\vee$
4	$\rightarrow$
5	$\leftrightarrow$

Creating larger truth Tables

Example:

Create a truth table for: $(p \wedge \sim q) \vee r$

For each variable, we have 2 possible truth values (T, F)

How many rows are there in the table?

Rule: for n variables, there are 2^n possible truth value combinations

Creating larger truth Tables

We have 3 variables (p, q, r)

The truth table must have $2^3 = 8$ rows

p	q	r	(p ∧ ~q) ∨ r
T	T	T	
T	T	F	
T	F	T	
T	F	F	
F	T	T	
F	T	F	
F	F	T	
F	F	F	

Creating a Truth Table

p	q	r	~q	p ∧ ~q	(p ∧ ~q) ∨ r
T	T	T	F		
T	T	F	F		
T	F	T	T		
T	F	F	T		
F	T	T	F		
F	T	F	F		
F	F	T	T		
F	F	F	T		

Creating a Truth Table

p	q	r	~q	p ∧ ~q	(p ∧ ~q) ∨ r
T	T	T	F	F	
T	T	F	F	F	
T	F	T	T	T	
T	F	F	T	T	
F	T	T	F	F	
F	T	F	F	F	
F	F	T	T	F	
F	F	F	T	F	

Creating a Truth Table

p	q	r	~q	p ∧ ~q	(p ∧ ~q) ∨ r
T	T	T	F	F	T
T	T	F	F	F	F
T	F	T	T	T	T
T	F	F	T	T	T
F	T	T	F	F	T
F	T	F	F	F	F
F	F	T	T	F	T
F	F	F	T	F	F

Activity

Build the truth table for the compound proposition:

$p \wedge (q \vee r)$

Activity

Build the truth table for the compound proposition:

$p \wedge (q \vee r)$

p	q	r	$q \vee r$	$p \wedge (q \vee r)$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	F	F
F	T	T	T	F
F	T	F	T	F
F	F	T	T	F
F	F	F	F	F

Application to Programming

C / C++ / Java:

- Negation: `if (!p) {}`
- Conjunction: `if (p && q) {}`
- Disjunction: `if (p || q) {}`

Example:

```
if (p && (q || r)) {
    ...
}
```

What about in Python?

Applications

Propositional Logic in Computer hardware circuits

Applications

Writing system and software Specification

e.g.: “The automated reply cannot be sent when the file system is full”

Logical equivalence

Compound propositions that have the same truth values in **all possible cases** are logically equivalent.

Notation:

$p \equiv q$ means that p and q are logically equivalent

Logical Equivalence

Given the statement: "It is cold and raining" $p \wedge q$

What if we said "It is raining and cold" $q \wedge p$

Are we saying the same thing?

p	q	$p \wedge q$	$q \wedge p$
T	T	T	T
T	F	F	F
F	T	F	F
F	F	F	F

Logical Equivalence

The values of $p \wedge q$ and $q \wedge p$ are the same for all truth table row.

The compound propositions $p \wedge q$ and $q \wedge p$ are logically equivalent.

$p \wedge q \equiv q \wedge p$

p	q	$p \wedge q$	$q \wedge p$
T	T	T	T
T	F	F	F
F	T	F	F
F	F	F	F

Example

Assume that we have two new compound propositions:

Let $X = p \longrightarrow q$ and $Y = \sim p \vee q$

Is $X \equiv Y$?

Activity

Assume that we have two new compound propositions:

Let $X = \sim p \vee \sim q$ and $Y = \sim(p \vee q)$

Is $X \equiv Y$?

Activity solution

Assume that we have two new compound propositions:

Let: $X = \sim p \vee \sim q$

$Y = \sim(p \vee q)$

Write the truth table

Observe the true values for X and Y

X is not equivalent to Y

De Morgan's Laws

Compare the results from the two truth tables:
Comparing the two tables, we will notice a pattern:
These properties are called De Morgan's Laws

p	q	~p ∧ ~q	~p ∨ ~q	~(p ∧ q)
T	T			
T	F			
F	T			
F	F			

De Morgan's Laws

The negation of a conjunction is the disjunction of the negations.

$\neg(p \land q) \equiv \neg p \lor \neg q$

The negation of a disjunction is the conjunction of the negations.

$\neg(p \lor q) \equiv \neg p \land \neg q$

Application to Programming

Recall our earlier example: $p \wedge (q \vee r)$

In C/C++/Java, this becomes

```
if (p && (q || r)) {}
```

Now, negate the proposition using De Morgan's Laws

```
if (!p || (!q && !r)) {}
```

Activity

We are given the following statement:

“The server is down or the application is running”

Using De Morgan’s Laws, write the negation of this statement.

Tautology and Contradiction

Tautology: compound proposition that is always true, regardless of truth values of its statement variables

Contradiction: compound proposition that is always false, regardless of truth values of its statement variables

Tautology and Contradiction

The compound proposition $p \vee \sim p$ is a tautology

The compound proposition $p \wedge \sim p$ is a contradiction

Notation:

tautology = t

contradiction = c

p	~p	$p \vee \sim p$	$p \wedge \sim p$
T	F	T	F
F	T	T	F

Logical Equivalence Laws

Statement variables: p, q, r | Tautology and contradiction: t, c

Commutative laws	$p \wedge q \equiv q \wedge p$	$p \vee q \equiv q \vee p$
Associative laws	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	$(p \vee q) \vee r \equiv p \vee (q \vee r)$
Distributive laws	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
Identity laws	$p \wedge t \equiv p$	$p \vee c \equiv p$
Negation laws	$p \vee \sim p \equiv t$	$p \wedge \sim p \equiv c$
Double negative law	$\sim(\sim p) \equiv p$	
Idempotent laws	$p \wedge p \equiv p$	$p \vee p \equiv p$
Universal bound laws	$p \vee t \equiv t$	$p \wedge c \equiv c$
De Morgan's laws	$\sim(p \wedge q) \equiv \sim p \vee \sim q$	$\sim(p \vee q) \equiv \sim p \wedge \sim q$
Absorption laws	$p \vee (p \wedge q) \equiv p$	$p \wedge (p \vee q) \equiv p$
Negations of t and c	$\sim t \equiv c$	$\sim c \equiv t$

Activity

Prove the following equivalence: $p \vee (p \wedge q) \equiv p$

Activity solution

Prove the following equivalence:

$p \vee (p \wedge q) \equiv p$

Let's use the Logical Equivalence Laws:

By the distributive law: $p \vee (p \wedge q) \equiv (p \vee p) \wedge (p \vee q)$

By Idempotence: $(p \vee p) \wedge (p \vee q) \equiv p \wedge (p \vee q)$

By the absorption law: $p \wedge (p \vee q) \equiv p$